

Dimensionality Reduction

Applied Machine Learning — Session 3, Chapter 3

The Curse of Dimensionality

As dimensions grow, data becomes increasingly **sparse**.

```
1D: ..... (10 points, close together)
2D: . . .   (10 points, more spread out)
    .       .
    .       .
100D: ???????? (10 points – vastly empty space)
```

Consequences:

- Distance-based algorithms fail
- Exponentially more data needed
- Visualization impossible beyond 3D
- Overfitting risk increases

The Blessing of Dimensionality

High-dimensional data often has low intrinsic dimensionality.

Example: millions of face photos

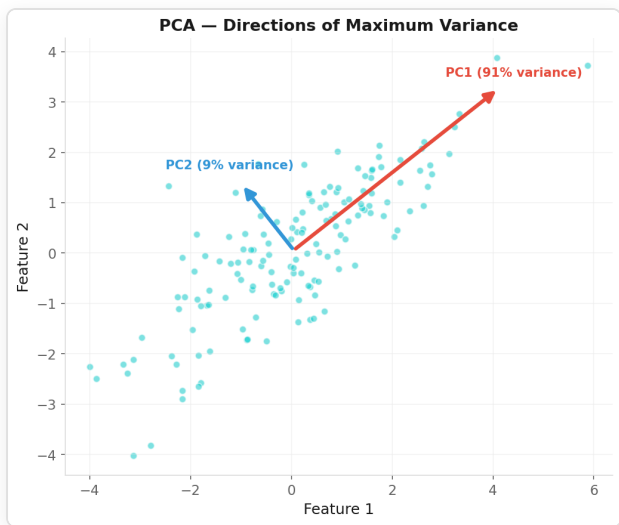
→ vary in expression, lighting, angle, identity

→ only ~50 "meaningful" dimensions

Dimensionality reduction finds this low-dimensional structure.

PCA: Principal Component Analysis

Find the directions of maximum variance.



- **PC1:** direction of most variance
- **PC2:** direction of most remaining variance, $\perp$ PC1
- Keep top k components $\rightarrow$ compression!

PCA in Code

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

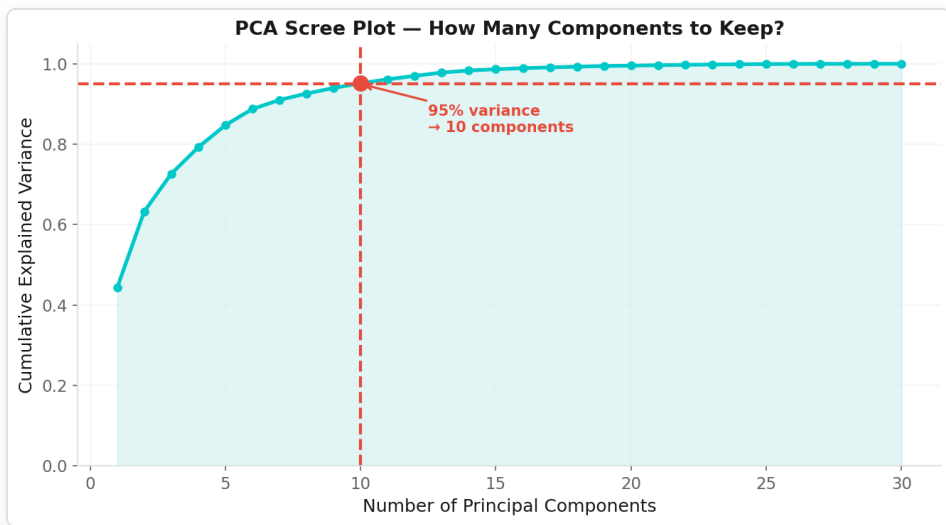
# Always scale before PCA!
pca_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components=2))
])

X_2d = pca_pipe.fit_transform(X)

# How much variance did we keep?
ratios = pca_pipe['pca'].explained_variance_ratio_
print(f'Variance explained: {ratios.sum():.1%}')
```

Choosing the Number of Components

Scree plot: plot cumulative explained variance vs. n_components



```
pca_full = Pipeline([('s', StandardScaler()), ('p', PCA())])  
pca_full.fit(X)
```

PCA Visualization

```
X_2d = pca_pipe.fit_transform(X)

plt.scatter(X_2d[:, 0], X_2d[:, 1], c=y, cmap='tab10')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('PCA: 2D projection of high-dimensional data')
```

Even 2 components often reveal clear class structure!

PCA as Preprocessing

```
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components=0.95)), # float = keep 95% variance
    ('clf', RandomForestClassifier())
])

pipeline.fit(X_train, y_train)
print('Components used:', pipeline['pca'].n_components_)
print('Accuracy:', pipeline.score(X_test, y_test))
```


t-SNE: Non-linear Visualization

t-Distributed Stochastic Neighbor Embedding

Key difference from PCA:

- Non-linear: reveals curved structure
- Preserves **local** similarity
- Great for visualization — **not for preprocessing**

```
from sklearn.manifold import TSNE

X_tsne = TSNE(
    n_components=2,
    perplexity=30,      # ~5-50, controls local vs global
    random_state=42
).fit_transform(X_scaled)
```

t-SNE: Common Mistake

"This t-SNE shows 5 clusters → the data has 5 natural groups."

Wrong! t-SNE cluster size and distance are NOT meaningful.

✅ **What t-SNE does well:**

- Shows local neighborhood structure
- Reveals which samples are similar to each other

❌ **What t-SNE does NOT show:**

- Global distances between clusters
- The actual number of clusters (perplexity changes this!)
- Anything useful for further ML steps

t-SNE: What You CAN and CANNOT Conclude

✓ Can:

- "These samples cluster together in t-SNE → probably similar"
- "This group looks isolated → might be a distinct class"

✗ Cannot:

- "Cluster A is 3× farther from B than C" — distances not meaningful
- "This t-SNE has 5 clusters → data has 5 groups" — perplexity matters

Never over-interpret t-SNE plots!

PCA vs t-SNE vs UMAP

	PCA	t-SNE	UMAP
Linear	✓	✗	✗
Use for ML prep	✓	✗	⚠
Use for visualization	✓	✓	✓
Speed	Fast	Slow	Medium
Global structure	✓	✗	⚠
Sklearn	✓	✓	pip install

Default choice: PCA for preprocessing, t-SNE/UMAP for visualization.

Now: Exercises!

→ Open `03-exercises/ch09_dimensionality_reduction_exercises.ipynb`

Task: Apply PCA to a high-dimensional dataset.

Visualize, choose components, use as preprocessing.

~10 minutes

Key Takeaways

- High dimensions → sparse data, distance fails, overfitting
- PCA: linear, preserves variance, for preprocessing AND visualization
- Always StandardScale before PCA
- Choose components that explain $\geq 95\%$ variance
- t-SNE: non-linear, local structure, visualization ONLY
- Never interpret t-SNE distances as meaningful

Next: Session 4

Reinforcement Learning

"What if the machine learns by trial and error – like us?"